

TECHNICAL WHITEPAPER

Eternal Beings Zcash Protocol Whitepaper

EBZ V0 Memo Protocol, Indexer, Renderer, and V0.5 Trading Protocol

Version 1.0

Publication date: September 9, 2026

eternalbeings.space

STATUS: IMPLEMENTABLE PROTOCOL DRAFT

Executive Summary

This whitepaper specifies an implementable first version of Eternal Beings on Zcash. V0 uses the encrypted 512 byte memo attached to a shielded Zcash output as a protocol carrier. A published protocol Unified Address and Unified Incoming Viewing Key allow independent indexers to discover and decrypt protocol messages. Each compliant indexer validates the same signatures, applies the same state machine, and reconstructs the same canonical Being state. A deterministic renderer then generates SVG artwork and metadata from that state.

V0.5 adds transfer, listing, cancellation, and purchase operations. It defines two settlement modes. Publicly verifiable settlement places a transparent ZEC payment and a shielded BUY memo in the same Zcash transaction, allowing an indexer to verify payment without custody. Shielded cooperative settlement preserves more payment privacy but requires seller acknowledgement and is not trustless atomic settlement.

MINT in V0 creates an application protocol identity. It does not issue a native Zcash custom asset and is not enforced by Zcash consensus. Zcash nodes confirm and order transactions; EBZ rules determine whether those transactions create or update a Being. This distinction is permanent in the V0 documentation and user interface.

The specification can be implemented on regtest and testnet now. It does not depend on Zcash Shielded Assets. As of the publication date, ZIP 226, ZIP 227, and ZIP 228 remain Draft, while ZIP 220 is Withdrawn. EBZ must not describe V0 as a consensus native NFT or a deployed ZSA.^{[9][10][11][12]}

1 Protocol Goals

EBZ defines a long lived digital entity that can be reconstructed without the official website. The protocol has the following goals.

- A Being has a unique identity, genome, state, history, and current control key.
- The Zcash payment address and source of funds do not need to be public.
- Every state transition is authorized by the current control key.
- MINT is not first come first served. A future block beacon ranks commitments after the entry window closes.
- Every protocol action requires its own Zcash transaction. A user cannot place hundreds of MINT or evolution actions in one memo to avoid per transaction cost.
- Independent indexers can reproduce the same state from public rules and chain data.
- The renderer is a deterministic pure function of canonical state.
- V0.5 states exactly which trade data is public and which settlement guarantees do not exist.

2 Scope and Non Goals

V0 does not create an ERC 721 equivalent inside Zcash consensus. A Being exists because the EBZ protocol recognizes a valid sequence of memo events. A single website database is insufficient. A production release requires a published specification, reference implementation, test vectors, state checkpoints, and at least two independently operated indexers.

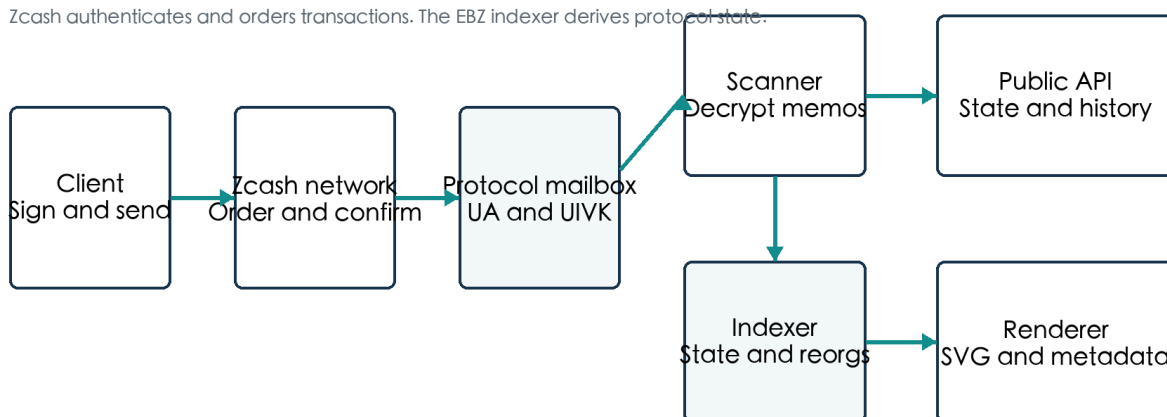
V0 does not hide every gameplay action. The memo is encrypted to the protocol mailbox, but the mailbox Unified Incoming Viewing Key is public so independent indexers can read the messages. The Zcash funding address can remain shielded, while Being identifiers, control public keys, operation types, and public state are visible through EBZ indexers. A hidden attribute can be represented by a commitment, but an indexer cannot validate the hidden value without an additional proof system.

V0.5 does not claim smart contract atomicity for a shielded asset sale. Publicly verifiable settlement is atomic only inside the EBZ state interpretation because the payment proof and BUY instruction share one Zcash transaction. Zcash consensus validates the ZEC payment but does not know that a Being exists. Shielded cooperative settlement requires seller acknowledgement and therefore introduces counterparty risk.

3 System Architecture

EBZ V0 Data Flow

Zcash authenticates and orders transactions. The EBZ indexer derives protocol state.



EBZ V0 System Architecture

Component	Responsibility	Authority boundary
User client	Generates control keys, builds memos, signs actions, and sends Zcash transactions	Controls only the user keys and transactions
Zcash network	Validates, orders, and confirms transactions	Determines the canonical transaction history
Protocol mailbox	Unified Address containing a shielded receiver	Receives outputs carrying EBZ memos
Scanner	Uses the published UIVK to scan and decrypt mailbox outputs	Discovers candidate events only
Indexer	Parses events, verifies signatures, executes rules, and handles reorganizations	Derives state from the public specification
Renderer	Converts canonical state into SVG and metadata	Determines only the specified visual representation

The recommended infrastructure is a Zebra full node, a Zaino or lightwalletd compatible compact block service, and a Rust scanner built on librustzcash. Z3 provides a combined Zebra, Zaino, and Zallet environment for regtest and deployment testing. Zallet is currently in beta and its RPC surface can change, so the EBZ client must isolate wallet specific calls behind a Wallet Adapter.[7][8][13]

4 World Manifest

Each EBZ world begins with an immutable [WorldManifest](#). The BLAKE2b-256 hash of its RFC 8785 canonical JSON is committed by the [world-genesis](#) event. An indexer must read and verify that event instead of trusting configuration copied from the official website.

Field	Type	Meaning
protocol	text	Fixed value ebz
protocol_major	uint8	0 for V0
protocol_minor	uint8	1 for the first release
network	uint8	0 mainnet, 1 testnet, 2 regtest
world_id	bytes16	Unique world identifier
genesis_height	uint32	First block scanned by an indexer
protocol_ua	text	Protocol mailbox Unified Address
protocol_uivk	text	Published Unified Incoming Viewing Key
carrier_value_zat	uint64	Minimum carrier output value, including zero
confirmation_depth	uint16	Application finality threshold, initially 10
commit_blocks	uint16	Length of each MINT commit window
beacon_delay_blocks	uint16	Distance from commit close to beacon block
reveal_blocks	uint16	Length of the reveal window
mint_slots_per_round	uint16	Maximum successful mints in one round
max_supply	uint32	Fixed maximum supply for the world
renderer_id	bytes32	Hash of the renderer source package or WASM
ruleset_id	bytes32	Hash of the state transition rules

A pilot world should use a deliberately small supply such as 333. A production world may set [max_supply](#) to 9999 before genesis. Once [world-genesis](#) is final, the value cannot be changed by editing a database or replacing the official service.

5 Zcash Memo Carrier

5.1 Network Constraints

A memo attached to a shielded Zcash output is exactly 512 bytes. It can be read by the recipient, the sender, or a party that receives suitable viewing capability.[1][2] EBZ uses the arbitrary binary form described by ZIP 302. Byte zero is `0xFF`; the remaining 511 bytes contain the EBZ envelope. EBZ must not use reserved lead byte values from `0xF6` through `0xFE`. [2]

The memo must be attached to a shielded output addressed to `protocol_ua`. Orchard is preferred. A memo cannot be attached to a transparent output. A ZIP 321 payment request can carry the protocol address, amount, and memo for compatible wallets.[5]

The reference transaction builder should use a zero zatoshi shielded carrier output when supported. ZIP 231 explicitly discusses additional potentially zero valued outputs used to carry memos, but a specific wallet implementation can still reject zero value requests.[14] If the selected wallet cannot build the output, the WorldManifest must define a positive `carrier_value_zat` that has been tested on regtest. The carrier amount is neither a sale payment nor a gameplay score.

Wallets should calculate fees under ZIP 317. The current conventional formula charges 5000 zatoshi per logical action with a minimum of two grace actions. EBZ clients must calculate the fee through the active wallet implementation instead of hard coding a permanent fee.[4]

5.2 EB-ZEC JSON Envelope

EB-ZEC is deliberately inscription-like. The protocol command is a compact JSON object with `p`, `op`, and `v` fields. The wire memo is:

```
memo[0]    = 0xFF
memo[1..N] = UTF8(JCS(message))
memo[N+1..511] = zero padding
```

`JCS` means RFC 8785 JSON Canonicalization Scheme. Binary values use unpadded base64url. Integers may not exceed $2^{53}-1$ so number serialization remains identical across common implementations. Duplicate keys, noncanonical JSON, invalid UTF-8, floating point numbers, unknown required fields, messages larger than 511 bytes, and nonzero padding are invalid. Exactly one EB-ZEC operation is allowed in one Zcash transaction.

Every controlled command contains the current Being identifier, previous event, monotonically increasing nonce, controller public key, and Ed25519 signature. The signature is calculated after removing `sig`.

```
unsigned = message with sig removed
digest = BLAKE2b-256("EBZ_SIGN_V1" || UTF8(JCS(unsigned)))
sig = Ed25519.Sign(controller_sk, digest)
```

The transaction identifier cannot be signed because it does not exist until construction completes. Replay protection comes from `world`, `being`, `prev`, `nonce`, single-use references, and canonical chain ordering.

5.3 Operation Registry

<code>op</code>	Release	Purpose
<code>world-genesis</code>	V0	Commits the immutable WorldManifest

op	Release	Purpose
mint-commit	V0	Enters one MINT round
mint-reveal	V0	Reveals one selected commitment
transfer	V0.5	Changes the Being controller
hunt-start	V0	Locks a Being into a scene
hunt-resolve	V0	Resolves an eligible hunt
mutate	V0	Attempts a deterministic mutation
consume-permit	V0	Authorizes one Being to consume another
devour	V0	Consumes an authorized object
fuse	V0	Fuses an authorized Being into a primary Being
list	V0.5	Creates or replaces a listing
cancel	V0.5	Cancels a listing
buy	V0.5	Pays and transfers a public listing
payment-ack	V0.5	Acknowledges cooperative shielded payment

The normative schemas, field types, signing procedure, and complete message examples are defined in the companion **EB-ZEC Memo Inscription Protocol v1.0**.

6 Control Keys and Protocol Ownership

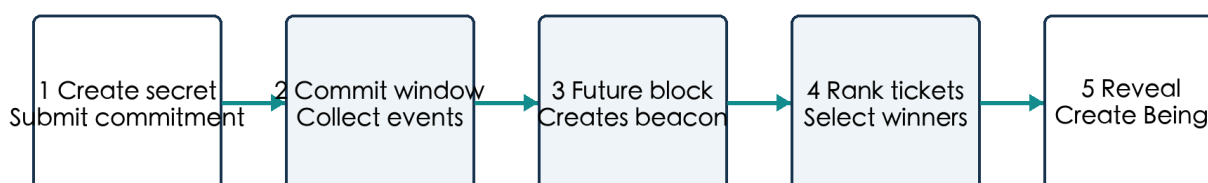
Shielded transactions do not reveal a sender address, and wallets do not expose one stable cross platform API for signing arbitrary application messages with a shielded spending key. EBZ therefore assigns an independent Ed25519 control key to each Being. The current `controller_pk` defines protocol ownership. The Zcash account funds fees and carrier outputs but does not directly define EBZ ownership.

The client creates the first control key during MINT. The private key remains on the user device and must be backed up. `transfer` replaces the current public key. Events signed by the former key are invalid after the transfer. A recipient should generate a new one time control key for every transfer to reduce linkability.

This design hides the direct relationship to a Zcash payment address, not the public action history of a Being. Publishing or reusing a control key can still link activity.

7 V0 MINT Protocol

MINT Commit Reveal Sequence



Unrevealed capacity returns to a later round. Every commitment requires its own Zcash transaction.

EBZ V0 MINT Sequence

7 1 Commit Reveal Rationale

Publishing a seed in the first transaction allows seed grinding and makes direct copying possible. A first come first served mint also rewards network position. EBZ separates entry, selection, and genome creation.

- COMMIT publishes only a commitment to secret data.
- The entry window closes before the randomness beacon exists.
- A future block hash ranks all valid commitments.
- Selected commitments reveal their secret and create a Being.

This mechanism does not prove that different public keys belong to different people. A permissionless protocol cannot prevent Sybil identities without identity, stake, or an additional cost. EBZ requires a separate Zcash transaction for every commitment and allows only one `mint-commit` operation per transaction. Batching hundreds of commitments into one memo is invalid.

7 2 MINT COMMIT

The client generates the following material locally.

```
controller_sk, controller_pk = Ed25519.KeyGen()
secret = RandomBytes(32)
salt = RandomBytes(32)
client = RandomBytes(16)
commitment = BLAKE2b-256(
    "EBZ_MINT_COMMIT_V1" ||
    world_id || round_id || controller_pk || secret || salt || client
)
```

The signed `mint-commit` message is:

```
{
  "p": "eb-zec",
  "op": "mint-commit",
  "v": 1,
  "world": "WORLD_ID_B64URL",
  "round": 42,
  "controller": "CONTROLLER_PUBLIC_KEY_B64URL",
  "commitment": "COMMITMENT_B64URL",
  "client": "RANDOM_16_BYTES_B64URL",
  "sig": "SIGNATURE_B64URL"
}
```

An indexer accepts the commitment only when all conditions hold.

- The event is inside the round commit height range.
- The envelope, network, version, signature, and canonical JSON are valid.
- The transaction contains only one `mint-commit`.
- The control public key has no other valid commitment in the round.
- The commitment and client value have not appeared before.
- Canonical supply remains below `max_supply`.

The client must encrypt and store `secret`, `salt`, `client`, `controller_sk`, round identifier, and commit event identifier. Losing this file makes reveal impossible.

7 3 Beacon and Selection

Let `Hc` be the final commit height. The beacon height is:

```
Hb = Hc + beacon_delay_blocks
beacon = block_hash(Hb)
ticket = BLAKE2b-256(
    "EBZ_MINT_TICKET_V1" || world_id || round_id || beacon || commit_event_id
)
```

All confirmed valid commitments are ordered by ascending `ticket`. A tie is broken by byte order of `commit_event_id`. The first `mint_slots_per_round` commitments are selected, limited by remaining supply.

A miner may have limited influence over one future block hash, and a participant can buy more chances by submitting independent commitments. V0 does not describe this mechanism as a verifiable random function. It is a compact selection method that prevents participants from knowing the result at commit time. A later protocol can combine several beacons or use a verifiable delay function.

7 4 MINT REVEAL

A selected controller submits a signed `mint-reveal` message that references the commitment event.

```
{
  "p": "eb-zec",
  "op": "mint-reveal",
  "v": 1,
  "world": "WORLD_ID_B64URL",
  "round": 42,
  "commit": "COMMIT_EVENT_ID_B64URL",
  "secret": "SECRET_B64URL",
}
```



```
{
  "salt": "SALT_B64URL",
  "controller": "CONTROLLER_PUBLIC_KEY_B64URL",
  "sig": "SIGNATURE_B64URL"
}
```

The indexer recomputes the commitment and ticket. Only a selected commitment can reveal, and it can reveal once. A missed reveal slot returns to the next round instead of moving to another participant in the same round. This avoids late ranking changes between indexer implementations.

7 5 Being Identity and Genome

```
being_id = BLAKE2b-256(
  "EBZ_BEING_ID_V1" || world_id || commit_event_id
)

genome = BLAKE2b-256(
  "EBZ_GENOME_V1" || world_id || being_id || secret || beacon
)

history_root_0 = BLAKE2b-256(
  "EBZ_HISTORY_V1" || reveal_event_id
)
```

The initial canonical state is:

```
{
  "being_id": "<32-byte hex>",
  "world_id": "<16-byte hex>",
  "controller_pk": "<32-byte hex>",
  "genome": "<32-byte hex>",
  "mass": 1,
  "complexity": 1,
  "power": 1,
  "skill": 1,
  "stage": 0,
  "devours": 0,
  "fusions": 0,
  "lineage_mask": "<derived uint16>",
  "nonce": 0,
  "last_event": "<reveal_event_id>",
  "history_root": "<32-byte hex>"
}
```

Lineage, initial glyph, palette seed, and morphology seed are derived only from [genome](#). An indexer and renderer must never add server randomness.

7 6 Low Level Zcash RPC

The preferred client uses `librustzcash` to construct an Orchard transaction. A `zcashd` 6.12 compatible adapter can use [z_sendmany](#), which supports Unified Addresses, hexadecimal memos, ZIP 317 fee calculation, and a [FullPrivacy](#) policy.[6]

```
zcash-cli z_sendmany \
  "<FROM_UNIFIED_ADDRESS>" \
  '[{"address": "<PROTOCOL_UA>","amount":<CARRIER_AMOUNT_ZEC>,"memo": "<1024_HEX_CHARS>"}]' \
  10 null "FullPrivacy"
```

The RPC returns an operation identifier. The adapter polls for completion.

```
zcash-cli z_getoperationstatus "[<operationid>]"
zcash-cli z_getoperationresult "[<operationid>"]
```

Production code must verify the returned txid and wait for [confirmation_depth](#). A wallet RPC must never be exposed directly to the public internet.

8 Reference Memo Commands

The EB-ZEC protocol command is the JSON message, not a particular CLI. A client signs, canonicalizes, and embeds the message in the encrypted memo. The examples below show the public

inscription syntax; the companion **EB-ZEC Memo Inscription Protocol v1.0** defines every field and validator rule.

8 1 MINT Commit and Reveal

```
{ "p": "eb-zec", "op": "mint-commit", "v": 1, "world": "WORLD_ID", "round": 42, "controller": "NEW_CONTROLLER_PK", "commitment": "COMMITMENT", "client": "RANDOM_16_BYTES", "sig": "SIGNATURE" }
```

After selection, reveal during the active reveal window:

```
{ "p": "eb-zec", "op": "mint-reveal", "v": 1, "world": "WORLD_ID", "round": 42, "commit": "COMMIT_EVENT_ID", "secret": "SECRET", "salt": "SALT", "controller": "NEW_CONTROLLER_PK", "sig": "SIGNATURE" }
```

8 2 Transfer

```
{ "p": "eb-zec", "op": "transfer", "v": 1, "world": "WORLD_ID", "being": "BEING_ID", "prev": "PREVIOUS_EVENT_ID", "nonce": 9, "to": "RECIPIENT_CONTROLLER_PK", "controller": "CURRENT_CONTROLLER_PK", "sig": "SIGNATURE" }
```

The **to** value is a fresh controller public key, not a Zcash payment address or viewing key hash. An incorrect key cannot be recovered by an administrator.

8 3 Hunt and Resolve

```
{ "p": "eb-zec", "op": "hunt-start", "v": 1, "world": "WORLD_ID", "being": "BEING_ID", "prev": "PREVIOUS_EVENT_ID", "nonce": 10, "scene": 3, "controller": "CURRENT_CONTROLLER_PK", "sig": "SIGNATURE" }
```

After the scene minimum duration:

```
{ "p": "eb-zec", "op": "hunt-resolve", "v": 1, "world": "WORLD_ID", "being": "BEING_ID", "prev": "HUNT_START_EVENT_ID", "nonce": 11, "controller": "CURRENT_CONTROLLER_PK", "sig": "SIGNATURE" }
```

8 4 Mutate

```
{ "p": "eb-zec", "op": "mutate", "v": 1, "world": "WORLD_ID", "being": "BEING_ID", "prev": "PREVIOUS_EVENT_ID", "nonce": 12, "controller": "CURRENT_CONTROLLER_PK", "sig": "SIGNATURE" }
```

8 5 Devour and Fuse

The sacrifice controller first signs a single-use permit naming the primary Being and either **devour** or **fuse** mode. This prevents an attacker from consuming somebody else's Being by writing its ID into a memo.

```
{ "p": "eb-zec", "op": "consume-permit", "v": 1, "world": "WORLD_ID", "being": "SACRIFICE_ID", "prev": "SACRIFICE_PREV", "nonce": 4, "consumer": "PRIMARY_ID", "mode": "fuse", "expires": 3115000, "controller": "SACRIFICE_CONTROLLER_PK", "sig": "SIGNATURE" }
```

The primary controller then references the permit:

```
{ "p": "eb-zec", "op": "fuse", "v": 1, "world": "WORLD_ID", "being": "PRIMARY_ID", "prev": "PRIMARY_PREV", "nonce": 22, "sacrifice": "SACRIFICE_ID", "permit": "PERMIT_EVENT_ID", "controller": "PRIMARY_CONTROLLER_PK", "sig": "SIGNATURE" }
```

devour uses the same fields with **op** set to **devour** and a permit whose mode is **devour**. Every action requires one signed memo in one separate Zcash transaction. Batch commands are invalid.

9 State Transitions

Every post mint Being event must satisfy all common rules.

- **being_id** exists and is active.
- **controller** equals the current control key for controller-authorized operations.
- **nonce** equals the current Being nonce plus one.

- `prev` equals the current `last_event`.
- The signature is valid.
- The operation body is canonical and within its allowed block range.

```

event_id = BLAKE2b-256(
  "EBZ_EVENT_V1" || txid || output_index || BLAKE2b-256(UTF8(JCS(message)))
)

history_root_next = BLAKE2b-256(
  "EBZ_HISTORY_STEP_V1" || history_root_prev || event_id
)

state_root_next = BLAKE2b-256(
  "EBZ_STATE_V1" || UTF8(JCS(state_next))
)

```

HUNT is divided into `hunt-start` and `hunt-resolve` and must cross the minimum duration defined by the ruleset. `mutate` derives its result from previous state, event identifier, and published probability tables. `devour` and `fuse` may consume only resources or Beings that the same EBZ world can verify and that supplied a valid single-use `consume-permit`. V0 cannot prove that an arbitrary Ethereum NFT was destroyed without an Ethereum lock proof, a trusted bridge, or a separate Veil specification.

10 Indexer Specification

10 1 Discovery and Ordering

The scanner starts at `genesis_height`, obtains compact blocks, and uses `protocol_uivk` to attempt decryption of outputs to the mailbox. A candidate enters parsing only if decryption succeeds, the memo is 512 bytes, and the EBZ prefix is valid.

Canonical event position is:

```
canonical_position = (block_height, transaction_index, output_index)
```

Indexers process events in this order. When two otherwise valid events attempt to consume the same state, the earlier canonical position wins. The losing event is recorded as rejected.

10 2 Confirmations and Reorganizations

Events below `confirmation_depth` are provisional. Events at or above the threshold are finalized for application use. This is a risk threshold, not absolute finality.[1]

The indexer stores block hashes, an event journal, and reversible state differences. A parent hash mismatch causes rollback to the nearest common ancestor followed by deterministic replay. API responses expose `observed_height`, `confirmations`, and `finality_status`.

10 3 State Checkpoints

Every 100 finalized blocks, an indexer computes:

```
checkpoint_root = MerkleRoot(
    sort_by_being_id(being_id || state_root)
)
```

At least two public indexers should publish the same root. An Ethereum anchor can add a public timestamp and dispute reference, but it must not be the sole source of EBZ correctness.

10 4 Invalid Events

The indexer stores txid, output index, block position, parser stage, error code, and raw memo hash for an invalid event. It does not expose decrypted memo material that is unrelated to required public fields.

11 Deterministic Renderer

The renderer is a pure function.

```
Render(canonical_state, renderer_id) -> canonical_svg_bytes
Metadata(canonical_state, renderer_id) -> canonical_json_bytes
```

The same state and renderer version must produce byte identical output across machines.

- Geometry uses integers or fixed point arithmetic.
- Every pseudo random branch derives from `BLAKE2b-256(genome || domain || index)`.
- The renderer never reads wall clock time, server randomness, network resources, or mutable configuration.
- SVG output contains no external URLs, scripts, fonts, or images.
- Animation uses only approved SVG animation and includes a static fallback.
- Attribute ordering, number precision, color encoding, and whitespace are canonical.

- `renderer_id` is the hash of the published source package or WASM bytes.

The recommended implementation is a Rust core compiled to WASM, shared by indexer, browser, and offline verification tools.

Method	Path	Response
GET	<code>/v1/beings/{being_id}/render.svg</code>	Canonical SVG
GET	<code>/v1/beings/{being_id}/metadata.json</code>	Canonical metadata
GET	<code>/v1/renderers/{renderer_id}</code>	Version, hash, and source location
POST	<code>/v1/render/verify</code>	State and output hash verification

12 Public Indexer API

Method	Path	Purpose
GET	<code>/v1/network</code>	Chain, synchronization height, and protocol version
GET	<code>/v1/worlds/{world_id}</code>	Manifest and supply state
GET	<code>/v1/mint/rounds/current</code>	Current round phase and block range
GET	<code>/v1/mint/rounds/{round_id}</code>	Commitments, beacon, ranking, and reveals
GET	<code>/v1/beings/{being_id}</code>	Current canonical state
GET	<code>/v1/beings/{being_id}/history</code>	Valid event history
GET	<code>/v1/events/{txid}/{output_index}</code>	Parse and confirmation status
GET	<code>/v1/checkpoints/{height}</code>	Checkpoint root
GET	<code>/v1/listings</code>	Active V0.5 listings

Construction endpoints never receive user private keys.

Method	Path	Purpose
POST	<code>/v1/prepare/mint-commit</code>	Returns unsigned canonical JSON and signing digest
POST	<code>/v1/prepare/mint-reveal</code>	Checks selection and returns signing material
POST	<code>/v1/prepare/transfer</code>	Returns TRANSFER signing material
POST	<code>/v1/prepare/list</code>	Returns LIST signing material
POST	<code>/v1/prepare/buy</code>	Returns BUY memo and multi output payment plan
POST	<code>/v1/validate/memo</code>	Validates a 512 byte memo without broadcasting
POST	<code>/v1/submit/raw-transaction</code>	Optional stateless broadcast relay

A prepare response contains `unsigned_json`, `memo_hex`, `memo_hash`, `signing_digest`, `expires_at_height`, `payments`, and a ZIP 321 URI. The server must never request a control private key, Zcash spending key, or wallet seed.

13 V0.5 Transfer Protocol

TRANSFER is signed by the current controller.

```
{
  "p": "eb-zec",
  "op": "transfer",
  "v": 1,
  "world": "WORLD_ID_B64URL",
  "being": "BEING_ID_B64URL",
  "prev": "PREVIOUS_EVENT_ID_B64URL",
  "nonce": 9,
  "to": "RECIPIENT_CONTROLLER_PUBLIC_KEY_B64URL",
  "controller": "CURRENT_CONTROLLER_PUBLIC_KEY_B64URL",
  "sig": "SIGNATURE_B64URL"
}
```

V0.5 uses immediate transfer. The client must display the Being identifier and recipient key fingerprint before signature. `to` is a controller public key rather than a Zcash address. The protocol has no administrator recovery.

14 V0.5 Listing Protocol

LIST body:

```
{
  "p": "eb-zec",
  "op": "list",
  "v": 1,
  "world": "WORLD_ID_B64URL",
  "being": "BEING_ID_B64URL",
  "prev": "PREVIOUS_EVENT_ID_B64URL",
  "nonce": 30,
  "price_zat": 25 000 000,
  "pay_to": "SELLER_TRANSPARENT_ZCASH_ADDRESS",
  "expires": 3 450 000,
  "controller": "SELLER_CONTROLLER_PUBLIC_KEY_B64URL",
  "sig": "SIGNATURE_B64URL"
}
```

This baseline listing uses publicly verifiable settlement. A separate cooperative shielded mode is described later. The `list` event identifier becomes `listing_id`.

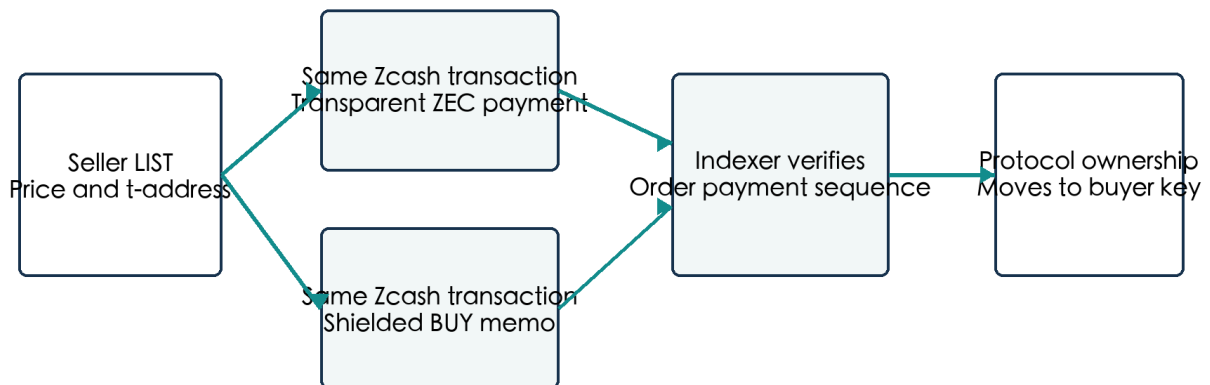
A new valid listing replaces an earlier listing for the same Being. Any event that changes `last_event` invalidates an existing listing. CANCEL references the current listing identifier and must be signed by the current controller.

Reference memo commands:

```
{"p":"eb-zec","op":"list","v":1,"world":"WORLD_ID","being":"BEING_ID","prev":"PREVIOUS_EVENT_ID","nonce":30,"price_zat":25 000 000,"pay_to":"SELLER_T_ADDRESS","expires":3 450 000,"controller":"SELLER_CONTROLLER_PK","sig":"SIGNATURE"}
{"p":"eb-zec","op":"cancel","v":1,"world":"WORLD_ID","being":"BEING_ID","prev":"LIST_EVENT_ID","nonce":31,"listing":"LIST_EVENT_ID","controller":"SELLER_CONTROLLER_PK","sig":"SIGNATURE"}
```

15 V0.5 Publicly Verifiable Buy

V0.5 Publicly Verifiable Settlement



EBZ V0.5 Public Settlement

The buyer constructs one Zcash transaction containing both items.

1. A transparent output pays at least `price_zat` to the seller address.
2. A shielded output to the protocol mailbox carries the BUY memo.

```
{
  "p": "eb-zec",
  "op": "buy",
  "v": 1,
  "world": "WORLD_ID_B64URL",
  "being": "BEING_ID_B64URL",
  "listing": "LIST_EVENT_ID_B64URL",
  "amount_zat": 25 000 000,
  "controller": "BUYER_CONTROLLER_PUBLIC_KEY_B64URL",
  "sig": "SIGNATURE_B64URL"
}
```

The indexer verifies the transparent output, amount, receiver, expiry, seller control, listing state, and buyer signature. The first valid BUY by canonical position wins. The controller changes to the buyer key without another seller transaction.

This is atomic at the EBZ state interpretation layer because payment evidence and the BUY instruction share one Zcash transaction. The payment amount and recipient are public. A buyer using shielded inputs can keep the funding address private, but the wallet must permit a revealed recipient policy.[6]

```
{
  "p": "eb-zec",
  "op": "buy",
  "v": 1,
  "world": "WORLD_ID",
  "being": "BEING_ID",
  "listing": "LIST_EVENT_ID",
  "amount_zat": 25 000 000,
  "controller": "BUYER_CONTROLLER_PK",
  "sig": "SIGNATURE"
}
```

16 V0.5 Shielded Cooperative Buy

Mode 1 pays a seller shielded address. A public indexer cannot independently prove that the seller received the correct amount unless the seller publishes a per order viewing key or payment disclosure. The baseline process is:

3. The buyer sends shielded payment and a BUY intent.
4. The seller wallet verifies receipt.
5. The seller sends `payment-ack` referencing the `buy` event and payment transaction.
6. The indexer transfers control after the acknowledgement is finalized.

This mode is noncustodial but not trustless atomic settlement. The seller can receive payment and refuse acknowledgement. The client must display that risk. A later design may use per order viewing capability, a PCZT workflow, or a deployed ZSA swap protocol.

17 Conflict Rules

- The first valid BUY for a listing wins.
- BUY and CANCEL in the same block are ordered by transaction index and output index.
- A filled listing returns `E_LISTING_FILLED` for later purchases.
- An expired listing or a listing whose Being state changed is invalid.
- Payment may exceed the price, but no protocol refund is created.
- One transparent payment output cannot satisfy multiple listings.
- A transaction may contain only one BUY memo.
- A chain reorganization rolls back EBZ ownership and listing state to the common ancestor.

18 Security Model

Risk	Effect	Required control
Memo replay	Repeats an old operation	Network, world, Being, nonce, previous event, and first valid ordering
Cross network replay	Reuses testnet data on mainnet	Network and world are signed
Front running	Copies a reveal or purchase	Reveal references a signed commitment; BUY binds listing and buyer key
Cheap batching	Avoids per action cost	One operation per memo and one MINT or BUY per transaction
Seed grinding	Selects favorable genomes	Commit before the future beacon exists
Sybil participation	One person uses many keys	Cannot be eliminated without identity or stake; each entry pays a transaction fee
Indexer disagreement	Different ownership results	RFC 8785 canonical JSON, fixed ordering, test vectors, and checkpoint roots
Chain reorganization	Reverses recent state	Provisional state, confirmation threshold, and journal rollback
Lost control key	Permanent loss of control	Encrypted backup; no administrator recovery
Compromised control key	Unauthorized transfer	One time keys and optional hardware signing
Malicious renderer	Misrepresents state	Renderer hash, pure function, and offline verification
Parser denial of service	Consumes resources	336 byte body limit, depth limit, integer bounds, and timeouts

Risk	Effect	Required control
Trade privacy leak	Reveals amount and seller receiver	Explicit settlement mode and pre transaction disclosure

The parser rejects duplicate JSON keys, noncanonical JCS encoding, floating point numbers, values outside integer bounds, unknown required fields, oversized messages, and nonzero padding. An implementation must not repair malformed input because different repair behavior would create state divergence.

19 Operations and Key Management

The scanner needs only the public UIVK. If carrier outputs are zero value, the protocol mailbox needs no operational spending key. If wallet compatibility requires a positive carrier value, the deployment documentation must state who can spend accumulated notes. A spendable mailbox introduces an operator privilege and cannot be described as fully autonomous.

The indexer database is not a key store. It contains the public UIVK, protocol fields, raw memo hashes, and derived state. Control private keys, Zcash spending keys, wallet seeds, and unrevealed salts never enter indexer APIs or logs.

Node RPC endpoints bind to localhost or a private network. Zcash security documentation warns that debug logging of wallet RPC activity can reveal shielded transaction details.[15]

20 Reference Implementation

```

ebz-protocol/
crates/
  ebz-types/      canonical JSON types and schemas
  ebz-crypto/     hashes and Ed25519 verification
  ebz-parser/     512-byte memo codec
  ebz-state/      deterministic transition engine
  ebz-scanner/    UIVK compact-block scan
  ebz-indexer/    reorg journal and database
  ebz-renderer/   fixed-point SVG renderer
  ebz-api/        REST service
  ebz-cli/        user commands
  ebz-wallet-adapter/ librustzcash and RPC adapters
specs/
  world-manifest.cddl
  memo-envelope.md
  state-machine.md
test-vectors/
  valid/
  invalid/
  reorg/
docker/
  regtest/
  testnet/

```

Rust is recommended. The scanner and wallet adapter use librustzcash, the renderer compiles to WASM, and PostgreSQL stores events and derived state. The state engine cannot depend on database query order; its only input is a canonically ordered event stream.

21 Testing and Acceptance

Area	Minimum acceptance condition
Memo codec	1000 random round trips; every invalid length, reserved prefix, and duplicate key is rejected
Signatures	Wrong network, world, key, or one bit mutation fails
MINT	Empty round, underfilled round, oversubscribed round, final supply boundary, missed reveal, and duplicate reveal
Ranking	Two independent implementations produce the same order for 10000 events
Reorganization	Roll back 1, 10, and 100 blocks and reproduce the original state root
State machine	Reject skipped nonce, stale previous event, concurrent transition, and overflow
Renderer	Linux, macOS, and browser WASM produce identical SVG hashes
API	Stable pagination, height pinned snapshots, and no decrypted memo leakage
V0.5	BUY versus CANCEL, underpayment, expiry, duplicate purchase, and reorganization
Recovery	Delete the database and rebuild the same checkpoint from genesis height

Before mainnet, V0 should run on testnet for at least 30 days, include an intentional reorganization exercise, and have a second implementation reproduce the checkpoint. Code that handles real ZEC payment verification, parsing, and signatures requires a focused security review even for a small project.

22 Release Plan

Phase One Protocol Freeze

Publish the WorldManifest schema, CDDL, memo codec, signature vectors, MINT rules, and CLI command contract. Complete automated regtest before opening a testnet world.

Phase Two State and Renderer

Add HUNT, MUTATE, DEVOUR, and FUSE. Publish the fixed point renderer and WASM package. Two independent indexers must continuously agree on checkpoints.

Phase Three Transfer

Enable TRANSFER and CANCEL. Test control key rotation, incorrect receiver handling, backup recovery, and reorganization behavior.

Phase Four Market

Enable publicly verifiable settlement first. Mark shielded cooperative settlement experimental and display its counterparty risk.

Phase Five ZSA Evaluation

Design one to one migration only after ZIP 226, ZIP 227, and ZIP 228 are deployed and supported by mainnet nodes, wallets, viewing tools, recovery workflows, and exchanges. Migration must preserve [being_id](#), [genome](#), [history_root](#), and fixed supply.

23 Versioning and Governance

The ruleset and renderer of a V0 world are fixed by hash. A bug fix that changes event validity cannot silently replace the old protocol implementation. It requires a new minor version, activation height, migration rule, and test vectors. A change to supply, identity, or canonical ordering requires a new [world_id](#).

An older indexer returns [E_UNSUPPORTED_OP](#) for an unknown operation. It never treats an unknown operation as a no op. New indexers retain the ability to replay old worlds. If the official service disappears, a world remains reconstructable from the manifest, public UIVK, Zcash chain, ruleset, and renderer.

24 Conclusion

EBZ V0 can launch without waiting for ZSA, but its integrity comes from an open deterministic protocol rather than native NFT enforcement by Zcash nodes. The practical implementation order is to freeze the memo and MINT rules, complete independent indexers and checkpoints, publish the deterministic renderer, and then open transfer and trading.

Publicly verifiable settlement is the appropriate first V0.5 market mode because an indexer can independently verify ZEC payment in the same transaction as the BUY memo. Fully shielded trading should wait for a verified settlement mechanism, a mature PCZT workflow, or deployed ZSA swaps.

Appendix A Error Codes

Code	Meaning
E_MEMO_LENGTH	Memo is not 512 bytes
E_MEMO_PREFIX	ZIP 302 prefix or <code>p</code> value is invalid
E_VERSION	Protocol version is unsupported
E_NETWORK	Network does not match
E_JSON_CANONICAL	Payload is not RFC 8785 canonical JSON
E_BODY_TOO_LARGE	JSON payload exceeds 511 bytes
E_SIGNATURE	Ed25519 signature is invalid
E_REPLAY	Nonce, commitment, or event was already used
E_NONCE	Nonce is not current nonce plus one
E_PREVIOUS_EVENT	Previous event is not the current head
E_NOT_CONTROLLER	Signer is not the current controller
E_MINT_PHASE	MINT event is outside the allowed phase
E_MINT_NOT_SELECTED	Commitment was not selected
E_MINT_EXPIRED	Reveal window expired
E_SUPPLY_CAP	World reached maximum supply
E_BEING_BUSY	Being is locked in an unresolved action
E_HUNT_EARLY	Hunt minimum duration has not elapsed
E_PERMIT_INVALID	Consume permit is missing, expired, stale, or has the wrong mode
E_SACRIFICE_CONSUMED	Sacrifice was already consumed
E_LISTING_STALE	Being changed after listing
E_LISTING_EXPIRED	Listing expired
E_LISTING_FILLED	Listing already sold
E_PAYMENT_AMOUNT	Payment is below the listed price
E_PAYMENT_RECEIVER	Payment receiver does not match
E_CONFLICT_LOST	Event lost canonical ordering conflict
E_UNSUPPORTED_OP	Operation name is unknown

Appendix B API Examples

Current MINT round:

```
{
```

```

"world_id": "a17c4e2d0 011 223 344 5 566 778 899aabb",
"round_id": 42,
"phase": "COMMIT",
"tip_height": 3 440 123,
"commit_start": 3 440 000,
"commit_end": 3 440 063,
"beacon_height": 3 440 069,
"reveal_end": 3 440 133,
"slots": 11,
"remaining_supply": 287,
"confirmation_depth": 10
}

```

Being response:

```

{
  "being_id": "83c1...9e20",
  "world_id": "a17c...aabb",
  "controller_fingerprint": "9D7A-41C2",
  "state": {
    "mass": 12,
    "complexity": 18,
    "power": 7,
    "skill": 5,
    "stage": 2,
    "devours": 3,
    "fusions": 1,
    "lineage_mask": 17,
    "nonce": 6
  },
  "state_root": "a8de...1f09",
  "history_root": "fe31...810a",
  "last_event": "9bc2...901d",
  "finality_status": "finalized",
  "confirmations": 18,
  "renderer_id": "76aa...88c0"
}

```

Appendix C Implementation Decisions

Decision	V0 choice	Reason
Chain carrier	Shielded output memo	Available now and encrypted to the recipient
Memo type	0xFF binary	ZIP 302 arbitrary data convention
Encoding	RFC 8785 canonical JSON	Inscription-like, readable, and deterministic across languages
Authorization	Independent Ed25519 control key	Does not depend on wallet arbitrary signing support
Selection	Commit plus future block hash	Reduces early seed selection and first come advantage
Finality	10 confirmations by default	Matches current wallet defaults and historical practice
State authority	Public rules plus multiple indexer roots	Reduces dependence on one database
Artwork	Fixed point deterministic SVG	Reproducible offline
V0.5 default trade	Transparent payment plus BUY memo	Indexer can verify payment independently
Native ZSA	Not required	Relevant ZIPs remain Draft

References

- [1] Zcash Protocol Specification. <https://zips.z.cash/protocol/protocol.pdf>
- [2] ZIP 302 Standardized Memo Field Format. <https://zips.z.cash/zip-0302>
- [3] ZIP 316 Unified Addresses and Unified Viewing Keys. <https://zips.z.cash/zip-0316>
- [4] ZIP 317 Proportional Transfer Fee Mechanism. <https://zips.z.cash/zip-0317>
- [5] ZIP 321 Payment Request URIs. <https://zips.z.cash/zip-0321>
- [6] Zcash 6.12.2 z_sendmany RPC. https://zcash.github.io/rpc/z_sendmany.html
- [7] Zcash Foundation Z3. <https://github.com/ZcashFoundation/z3>
- [8] Zallet. <https://github.com/zcash/zallet>
- [9] ZIP 226 Transfer and Burn of Zcash Shielded Assets. <https://zips.z.cash/zip-0226>
- [10] ZIP 227 Issuance of Zcash Shielded Assets. <https://zips.z.cash/zip-0227>
- [11] ZIP 228 Asset Swaps for Zcash Shielded Assets. <https://zips.z.cash/zip-0228>
- [12] ZIP 220 Zcash Shielded Assets Withdrawn. <https://zips.z.cash/zip-0220>
- [13] lightwalletd. <https://github.com/zcash/lightwalletd>
- [14] ZIP 231 Memo Bundles. <https://zips.z.cash/zip-0231>
- [15] Zcash Security Warnings. <https://zcash.github.io/zcash/user/security-warnings.html>
- [16] Eternal Beings The Shielded World Concept Document. September 2026.